

Coder avec un agent

Les bases, la méthode, puis l'usine.

- Syllabus · 15 modules · 42 heures
- Cours autonome + animation premium

Comprendre ce qui change. Prouver ce qui marche. Savoir reprendre.

Un parcours, deux usages

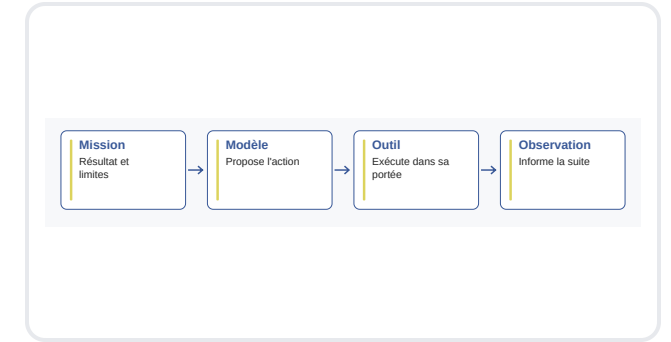
- Seul : lire, essayer, conserver ses preuves, comparer au corrigé.
- En groupe : démonstrations, binômes et retours individualisés.
- Même exigence : aucun résultat annoncé sans observation.

Le laboratoire est local, avec données fictives et identités simulées.

De l'assistant à l'agent

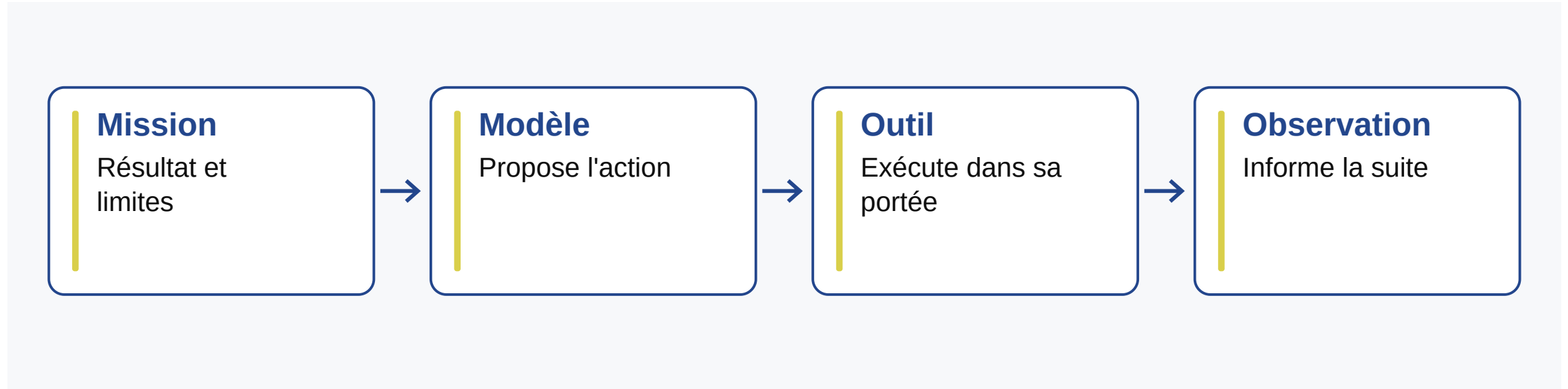
Une réponse convaincante n'est pas une action vérifiée.

- Le modèle génère à partir d'un contexte limité.
- Les outils lisent, écrivent ou exécutent.
- Le responsable fixe les pouvoirs et vérifie les effets.



Distinguer génération, outils, contexte, autonomie et responsabilité.

L'agent agit dans un environnement



Une capacité technique ne remplace ni l'autorisation ni la pertinence de l'action.

Un exemple à examiner

« Les tests passent. »

Quelle commande ? Dans quel dossier ?

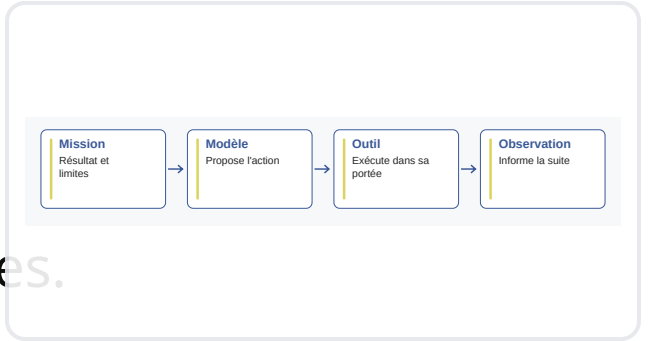
Quel résultat ? Sur quel état du code ?



Prédire → observer → expliquer l'écart.

À vous de produire

- E01 · Classer quatre situations d'assistance.
- E02 · Auditer une affirmation de tests réussis.
- Rendre une carte des pouvoirs et les observations manquantes.

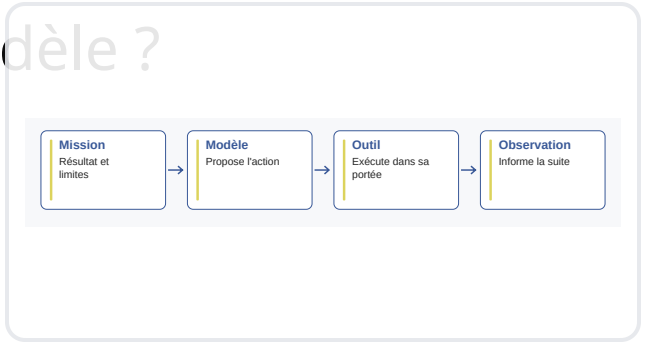


E01 + E02 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Un fichier présent sur disque est-il forcément connu du modèle ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

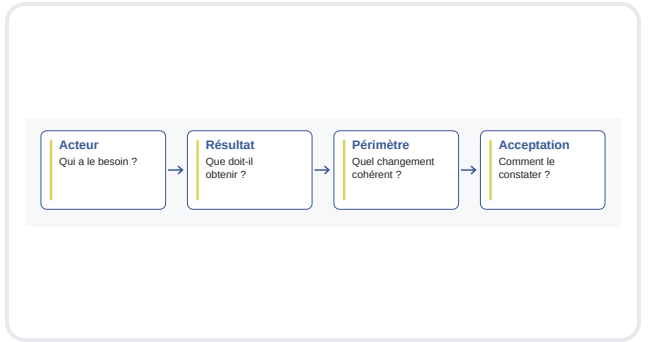


La reprise suivante s'appuie sur ce résultat conservé.

Transformer une demande en unité de travail

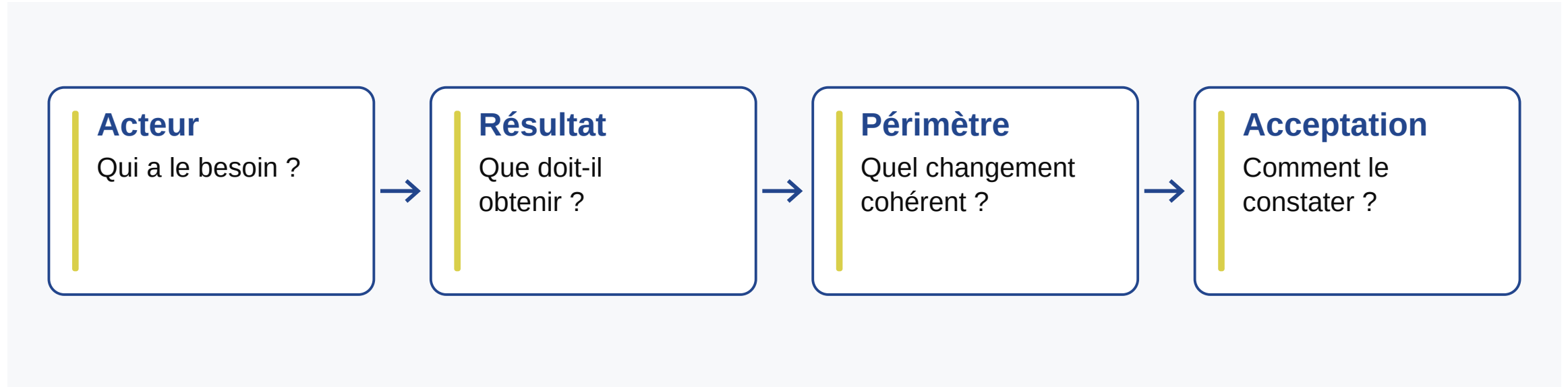
« Améliore l'application » ne dit pas quand s'arrêter.

- Nommer l'acteur et son résultat.
- Délimiter le changement et les exclusions.
- Décrire un succès, un refus et la donnée préservée.



Délimiter un résultat utile, observable et réalisable.

Passer de l'ambition à une unité



Une inconnue produit devient une décision explicite avant l'implémentation.

Un exemple à examiner

Bob crée une demande.

Titre invalide → refus, aucune insertion.

Titre valide → visible et conservé.

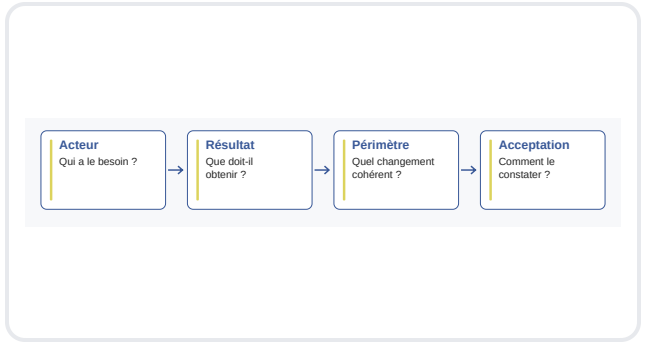
Hors périmètre : notifications et statistiques.



Prédire → observer → expliquer l'écart.

À vous de produire

- E03 · Découper une demande trop large.
- E04 · Décider sans élargir le mandat.
- Rendre une unité et un relevé de décisions explicites.

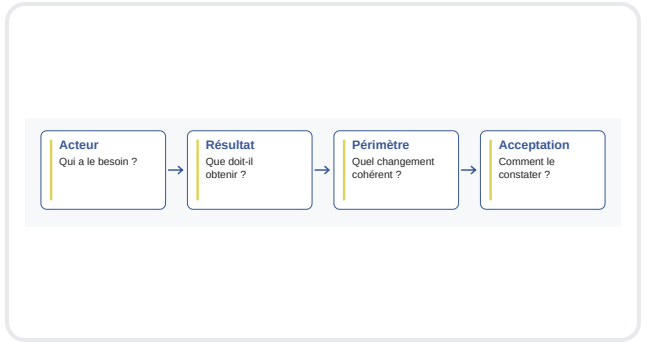


E03 + E04 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Une unité de travail correspond-elle à un fichier ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

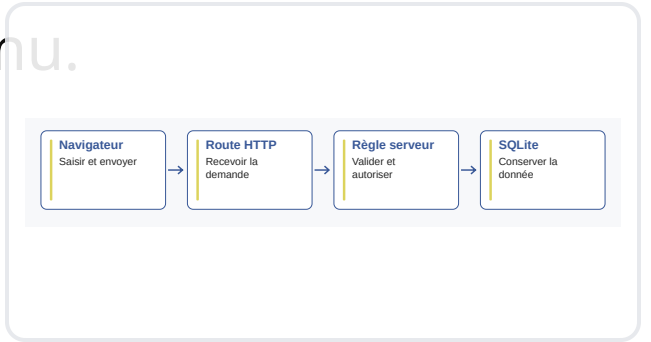


La reprise suivante s'appuie sur ce résultat conservé.

Construire le contexte

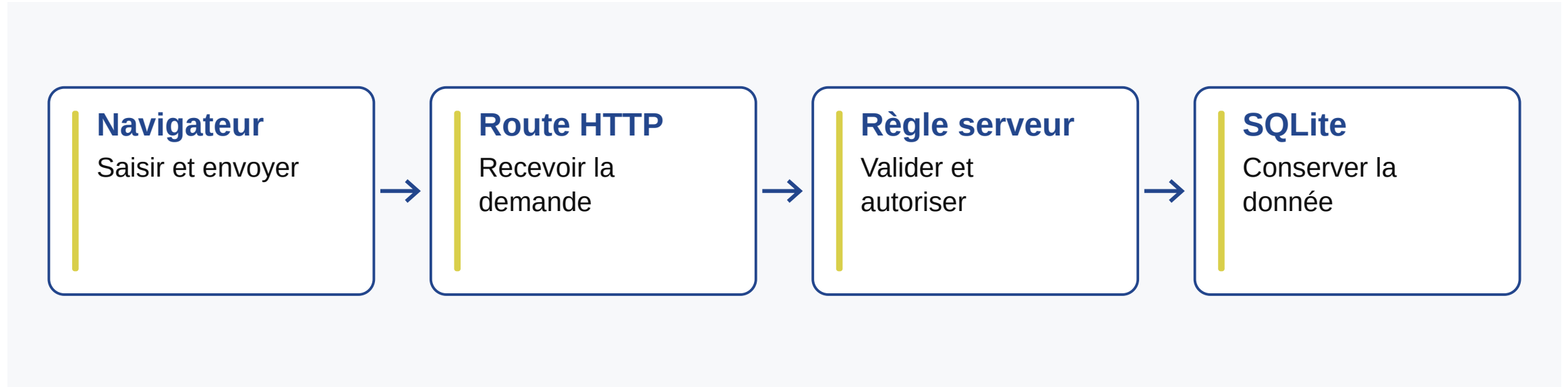
Une jolie interface peut masquer un fonctionnement inconnu.

- Partir d'un geste réellement observé.
- Suivre routes, règles et données.
- Séparer faits, hypothèses et décisions.



Explorer les sources et suivre le fonctionnement réel.

Suivre un geste de bout en bout



Le retour visible doit correspondre à l'état réellement conservé.

Un exemple à examiner

Accueil → Bob → Nouvelle demande

Titre : trois espaces → Créer

Attendu : refus sans nouvelle ligne.

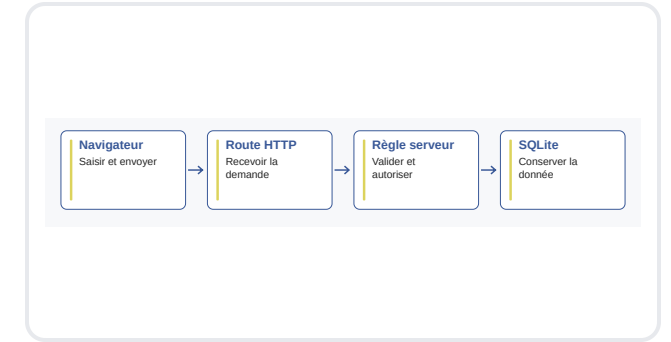
Départ : un titre vide est conservé.



Prédire → observer → expliquer l'écart.

À vous de produire

- E05 · Cartographier une création de bout en bout.
- E06 · Reproduire un défaut et traiter une donnée hostile.
- Rendre une carte sourcée et une fiche rejouable.

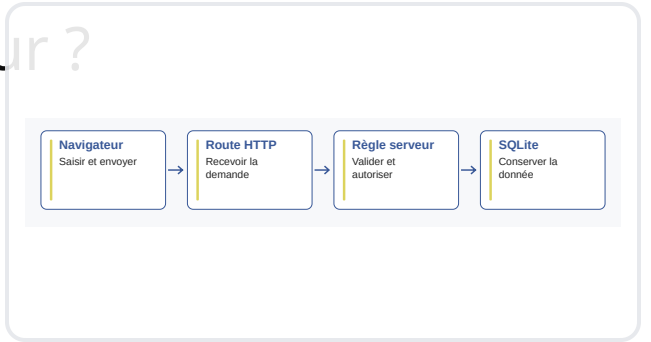


E05 + E06 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Recharger la page prouve-t-il la survie après arrêt du serveur ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.



La reprise suivante s'appuie sur ce résultat conservé.

Externaliser la mémoire

Une nouvelle session ne doit pas deviner la suite.

- Donner une responsabilité à chaque document.
- Conserver la décision, son motif et l'état réel.
- Tester la passation sans historique de chat.



Conserver les décisions et permettre la reprise d'une session.

La mémoire appartient au projet

README

Comment exécuter ?

DAT

Comment cela fonctionne ?

BACKLOG

Que reste-t-il à faire ?

JOURNAL

Pourquoi et où reprendre ?

Le CHANGELOG distingue changement présent et publication effectivement vérifiée.

Un exemple à examiner

Unité : DEM-01 · état : en cours

Référence : votre vrai commit de départ

Observé : titre vide encore accepté

Suite : préciser le contrat, puis corriger



Prédire → observer → expliquer l'écart.

À vous de produire

- E07 · Répartir les informations entre documents.
- E08 · Faire reprendre le travail sans le chat.
- Rendre les fichiers et une passation liée à un vrai état Git.



E07 + E08 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Quel document explique pourquoi la troncature a été refusée ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.



La reprise suivante s'appuie sur ce résultat conservé.

Séparer méthode globale et contexte local

Une méthode réutilisable ne connaît pas votre port local.

- Global : responsabilités et manière de travailler.
- Local : métier, commandes, données et contraintes.
- Le mécanisme de lecture doit être explicite.



Classer les règles et faire lire les bonnes sources.

Méthode partagée, contexte local

Global

Décider, prouver, rendre compte

Local

Données, commandes et métier

CLAUDE.md / CLAUDE_PROJECT.md · DESIGN_SYSTEM / DESIGN_SYSTEM_APP

Un exemple à examiner

AGENTS.md → lire CLAUDE.md

Puis CLAUDE_PROJECT.md, s'il existe.

Global ≠ installation globale du logiciel.

Un nom conventionnel ne garantit pas la lecture.



Prédire → observer → expliquer l'écart.

À vous de produire

- E09 · Classer la méthode et le contexte.
- E10 · Construire et contrôler le routage de lecture.
- Rendre un contrat court et son compagnon local.

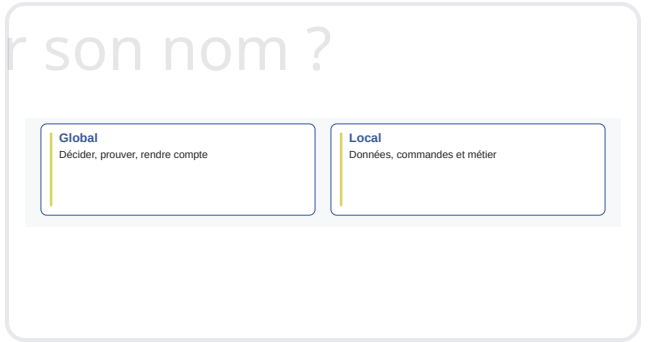


E09 + E10 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

CLAUDE_PROJECT.md est-il automatiquement découvert par son nom ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

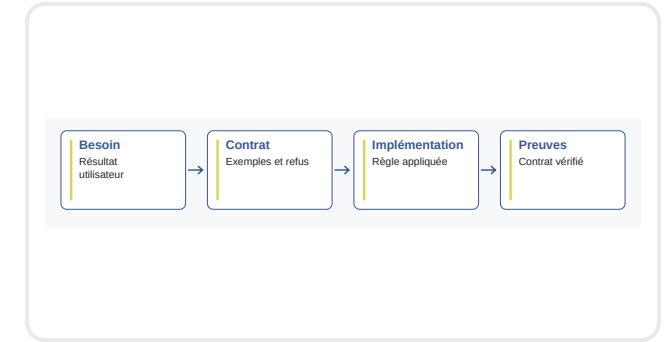


La reprise suivante s'appuie sur ce résultat conservé.

Spécifier avant d'implémenter

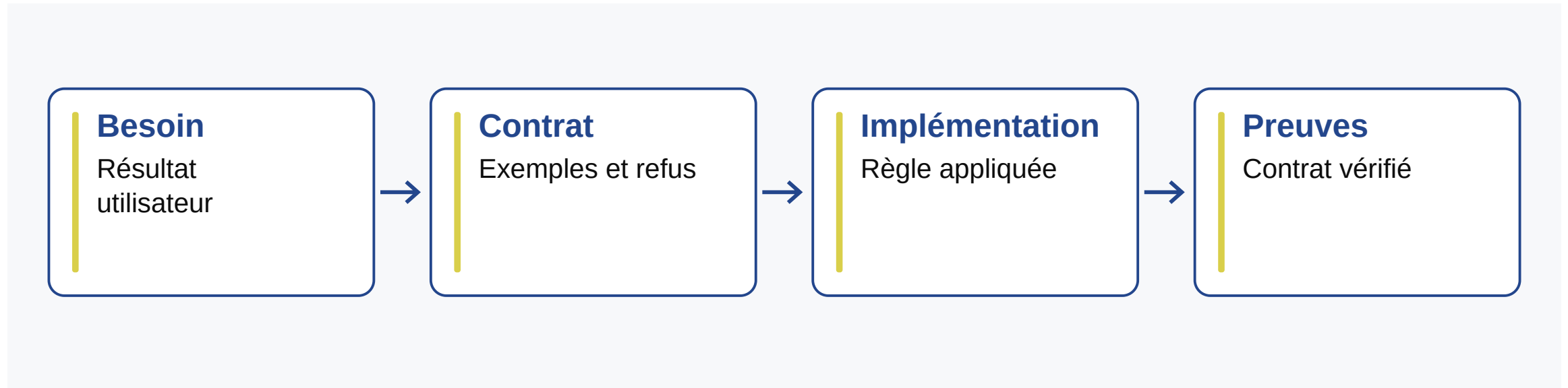
« Un titre propre » possède trop d'interprétations.

- Écrire les règles avant leur réalisation.
- Utiliser les bornes et les refus comme exemples.
- Relier contrat, unité, code et preuve.



Relier besoin, exemples, backlog, code et tests.

Relier besoin, code et preuve



La documentation suit le comportement ; les références doivent rester pertinentes.

Un exemple à examiner

Titre texte, espaces de bord retirés.

Longueur autorisée : 3 à 80 inclus.

2 → refus • 3 → accepté

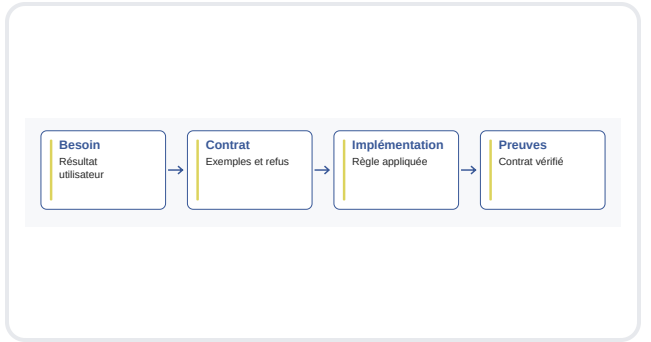
80 → accepté • 81 → refus



Prédire → observer → expliquer l'écart.

À vous de produire

- E11 · Rédiger le contrat des titres.
- E12 · Établir la matrice d'accès et la traçabilité.
- Rendre deux unités verticales avec leurs preuves attendues.

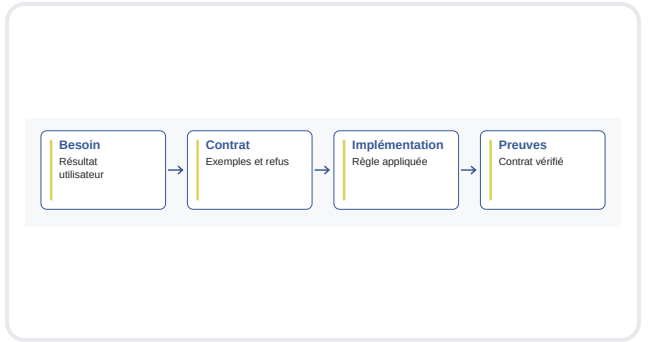


E11 + E12 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Pourquoi tester 2, 3, 80 et 81 plutôt que seulement 20 ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

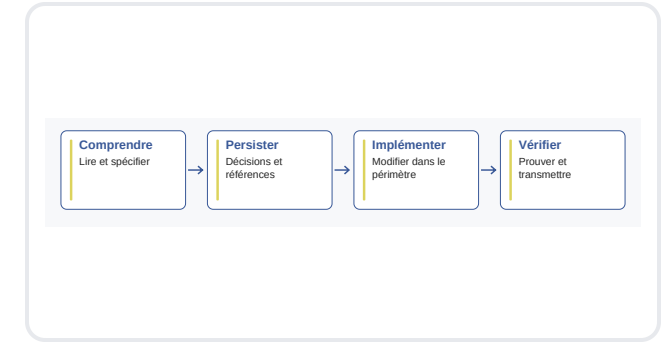


La reprise suivante s'appuie sur ce résultat conservé.

Concevoir la boucle d'exécution

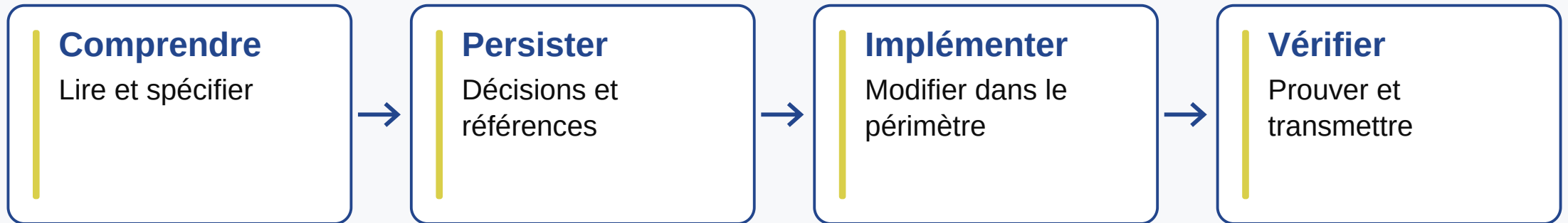
Un checkpoint n'est pas une livraison.

- Comprendre et persister avant d'implémenter.
- Vérifier puis corriger la cause du défaut.
- Transmettre l'état réel, y compris les preuves manquantes.



Ordonner l'exploration, la décision, l'implémentation et les preuves.

La boucle d'exécution d'une unité



Un défaut renvoie à sa cause ; un checkpoint ne signifie pas unité terminée.

Un exemple à examiner

Lire → décider → conserver

Implémenter → relire → prouver

Défaut → retour à la cause

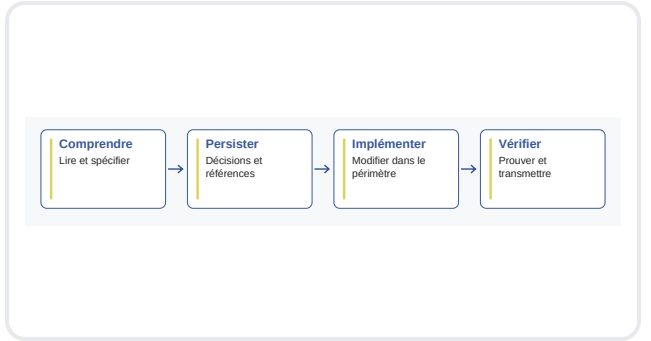
Fin → preuves + documents + état durable



Prédire → observer → expliquer l'écart.

À vous de produire

- E13 · Ordonner une session et ses checkpoints.
- E14 · Refuser une fausse clôture.
- Rendre un protocole et une définition de fin vérifiable.



E13 + E14 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Une capture enregistrée mais jamais ouverte est-elle une observation ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.



La reprise suivante s'appuie sur ce résultat conservé.

Rendre l'environnement reproductible

« Ça marche chez moi » décrit un poste, pas une procédure.

- Nommer runtime et dépendances.
- Documenter configuration, services et données.
- Rejouer sans remplacer un état existant.



Préparer runtime, configuration, services et données déterministes.

Ce qu'il faut pour reproduire

Runtime

Version et dépendances

Configuration

Paramètres documentés

Services

Démarrage et disponibilité

Données

Schéma, seed et état initial

Une procédure importante doit être rejouable depuis le README.

Un exemple à examiner

Code + Python + paramètres

Base neuve → schéma + trois demandes

Même base → données conservées

Nouveau nom → autre expérience isolée



Prédire → observer → expliquer l'écart.

À vous de produire

- E15 · Rejouer le bootstrap et la persistance.
- E16 · Diagnostiquer sans masquer la panne.
- Rendre un README rejouable et un diagnostic par couche.

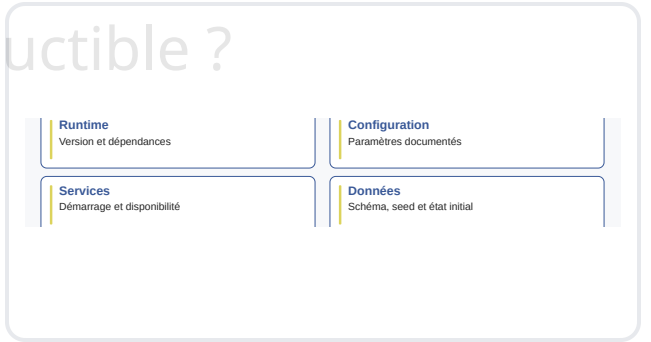


E15 + E16 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Un conteneur suffit-il à rendre tout l'environnement reproductible ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

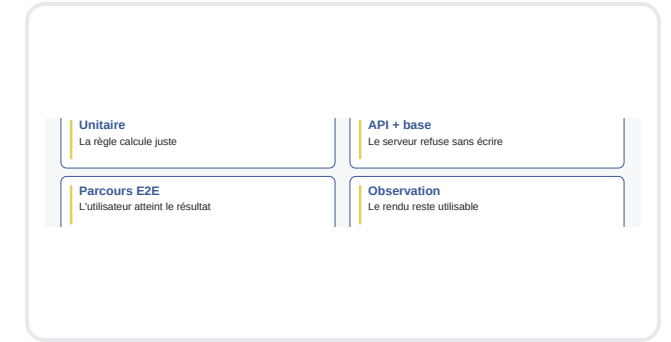


La reprise suivante s'appuie sur ce résultat conservé.

Construire la preuve

Un seul voyant vert ne prouve pas toute l'application.

- Unitaire : la règle isolée.
- API et base : la route et les effets.
- Parcours et observation : le résultat utilisable.



Produire et interpréter les tests et observations utiles.

Une règle, plusieurs regards

Unitaire

La règle calcule juste

API + base

Le serveur refuse sans écrire

Parcours E2E

L'utilisateur atteint le résultat

Observation

Le rendu reste utilisable

Un succès à un niveau ne dispense pas des autres preuves applicables.

Un exemple à examiner

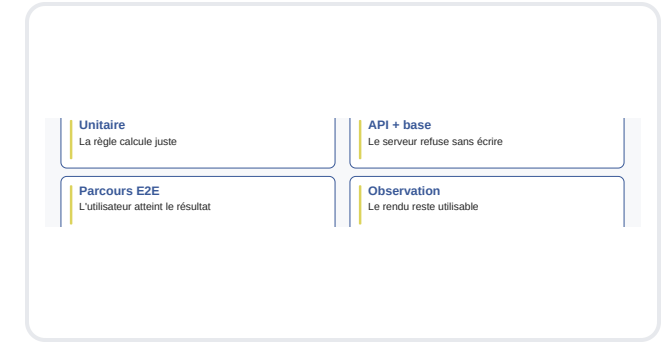
Avant DEM-01 : quatre tests rouges.
Corriger uniquement le contrat des titres.
Après : ses preuves passent.
Les défauts de droits restent visibles.

Unitaire La règle calcule juste	API + base Le serveur refuse sans écrire
Parcours E2E L'utilisateur atteint le résultat	Observation Le rendu reste utilisable

Prédire → observer → expliquer l'écart.

À vous de produire

- E17 · Corriger DEM-01 avec un rouge utile.
- E18 · Prouver le parcours et inspecter le rendu.
- Rendre les résultats, captures observées et limites.



E17 + E18 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Un test rouge d'import démontre-t-il le défaut métier recherché ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

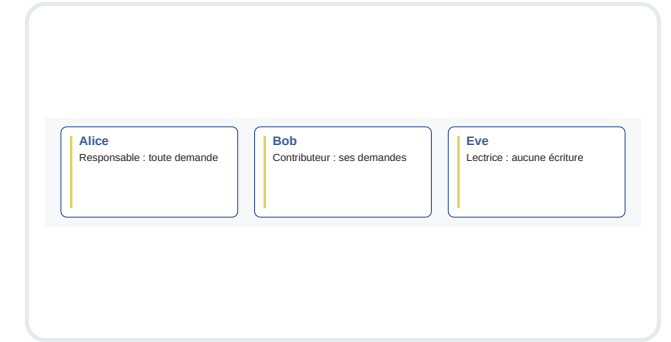
Unitaire La règle calcule juste	API + base Le serveur refuse sans écrire
Parcours E2E L'utilisateur atteint le résultat	Observation Le rendu reste utilisable

La reprise suivante s'appuie sur ce résultat conservé.

Encadrer sécurité et production

Cacher un bouton ne protège pas une écriture.

- Résoudre l'acteur côté serveur.
- Croiser rôle et propriété de la demande.
- Refuser avant d'écrire ; relire l'état après refus.



Appliquer les droits serveur et encadrer les opérations sensibles.

Qui peut clore cette demande ?

Alice

Responsable : toute demande

Bob

Contributeur : ses demandes

Eve

Lectrice : aucune écriture

Un refus est prouvé par sa réponse ET par l'absence de mutation.

Un exemple à examiner

Alice : peut clore toute demande.

Bob : peut clore uniquement les siennes.

Eve : aucune écriture.

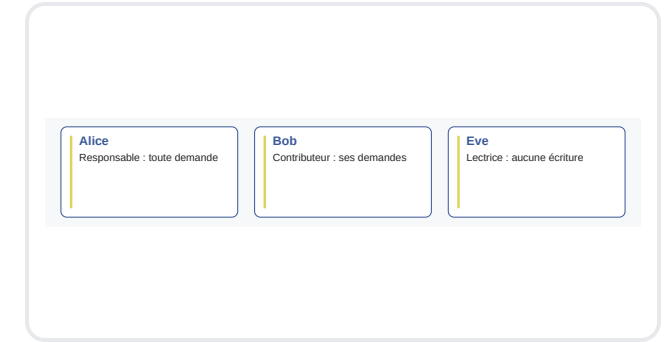
Bob → demande d'Alice : 403 ET état inchangé.



Prédire → observer → expliquer l'écart.

À vous de produire

- E19 · Identifier les valeurs non fiables.
- E20 · Corriger DEM-02 et préparer le retour arrière.
- Rendre les sept tests verts et le parcours multi-profils.

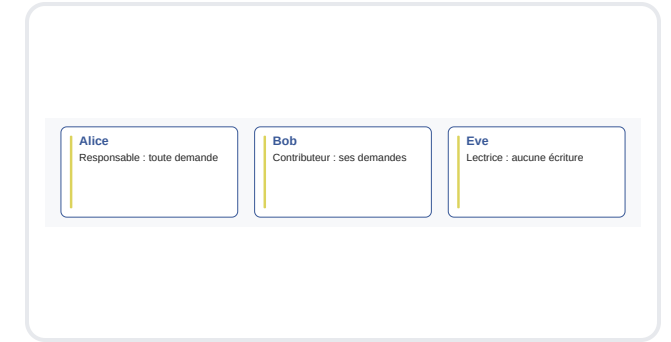


E19 + E20 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Pourquoi lire la ressource après une réponse 403 ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

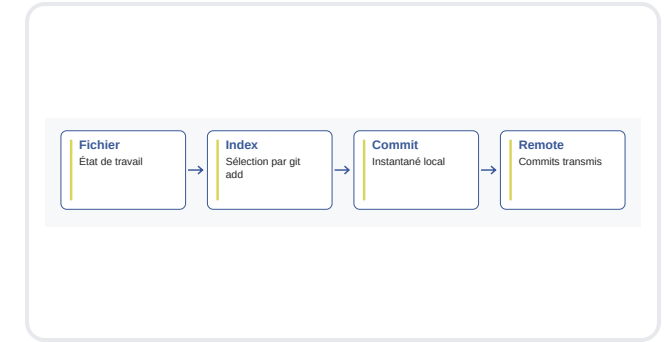


La reprise suivante s'appuie sur ce résultat conservé.

Utiliser Git comme mémoire durable

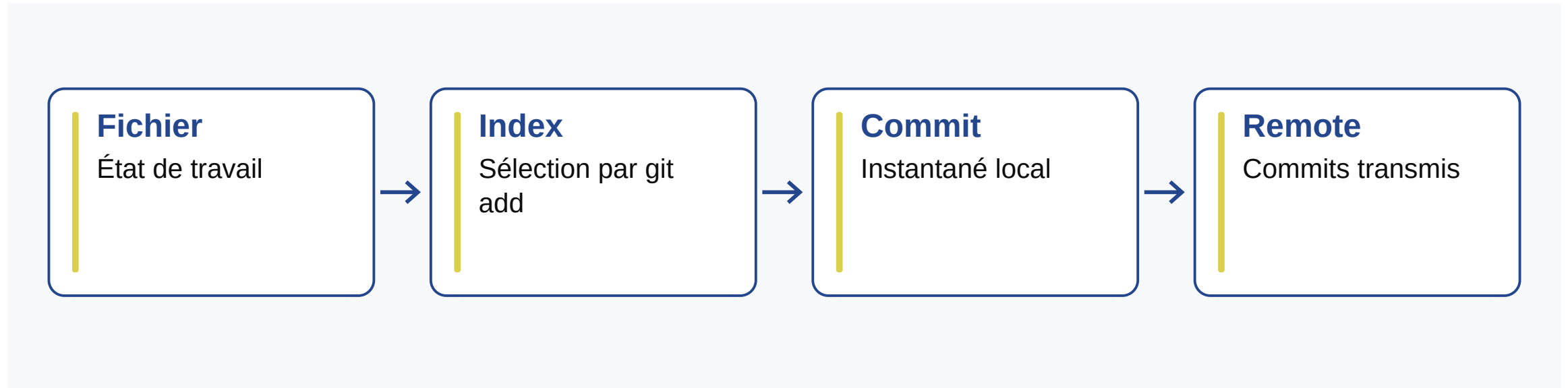
Le fichier ouvert n'est pas forcément le prochain commit.

- Travail, index, commit et remote sont distincts.
- Relire la sélection avant de la conserver.
- Fetch observe ; l'intégration modifie ; push transmet.



Lire, conserver, synchroniser et récupérer un changement.

Quatre lieux, quatre états



git diff examine le travail ; git diff --cached examine la sélection.

Un exemple à examiner

Écrire « v1 » → `git add`

Écrire « v2 » → `git diff`

Le prochain commit contient encore « v1 ».

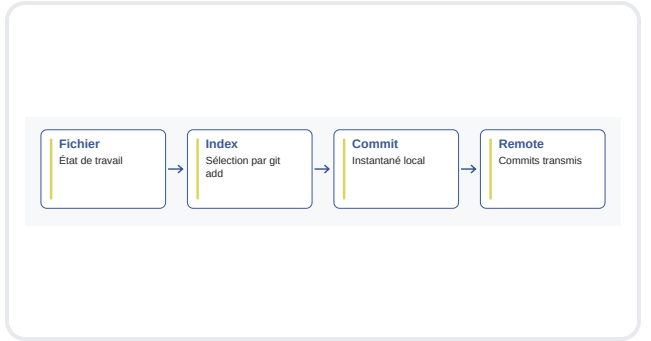
`git diff --cached` montre la sélection.



Prédire → observer → expliquer l'écart.

À vous de produire

- E21 · Observer l'index et créer un checkpoint.
- E22 · Relire un changement qui dépasse la mission.
- Rendre le vrai hash, le diff retenu et les refus argumentés.

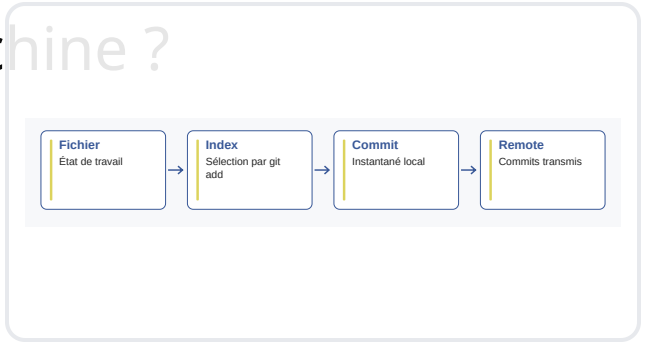


E21 + E22 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Un commit local protège-t-il de la perte complète de la machine ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

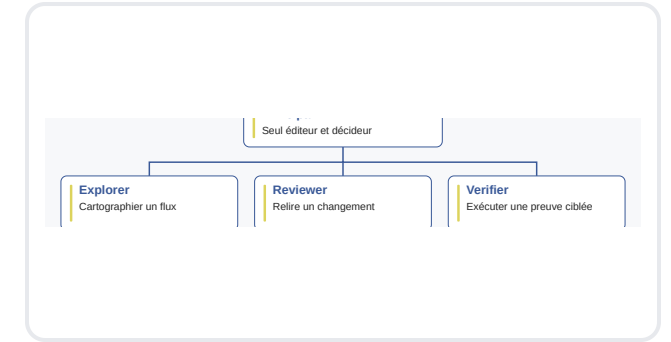


La reprise suivante s'appuie sur ce résultat conservé.

Orchestrer plusieurs agents

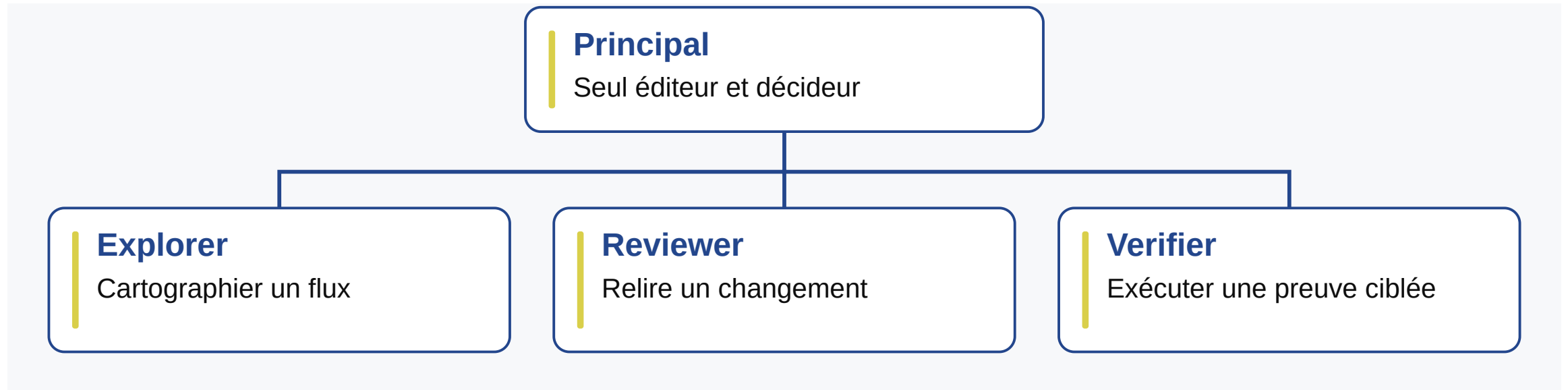
Plusieurs agents ne créent pas plusieurs responsables.

- Le principal décide et reste seul éditeur.
- Une mission déléguée est indépendante et bornée.
- Un résultat doit être vérifié avant synthèse.



Attribuer des missions bornées avec une responsabilité unique.

Un responsable, des missions bornées



Le vérificateur compare l'état avant/après ; aucune édition concurrente.

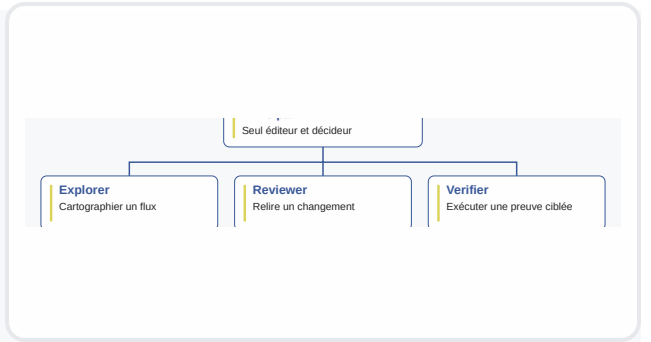
Un exemple à examiner

Explorer : quel est le flux réel ?

Reviewer : quel défaut contre le contrat ?

Verifier : quelle preuve sur quel état ?

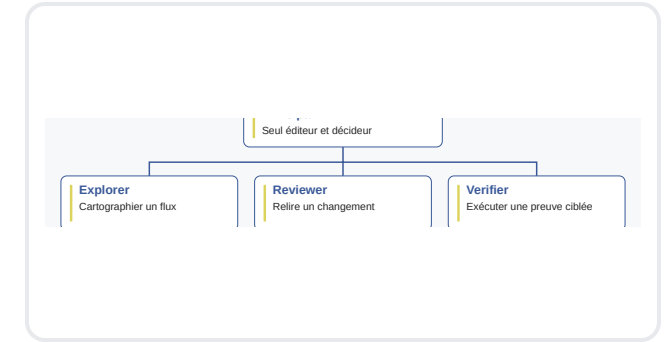
Principal : vérifier, décider, intégrer.



Prédire → observer → expliquer l'écart.

À vous de produire

- E23 · Rédiger une mission de revue bornée.
- E24 · Vérifier sans édition concurrente.
- Rendre contrat, observations et traitement d'une anomalie.

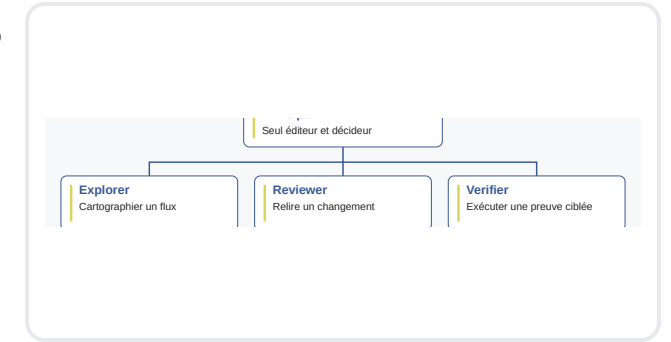


E23 + E24 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Qui tranche si le reviewer propose un autre choix produit ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

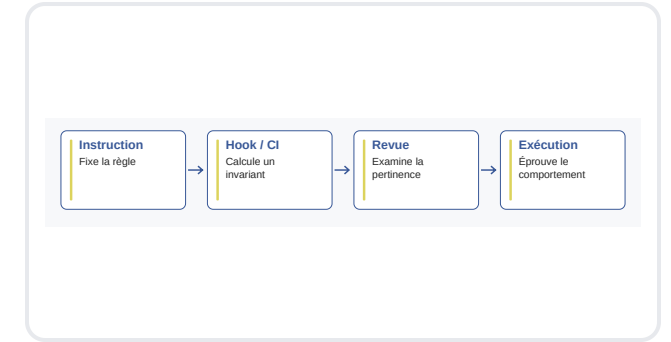


La reprise suivante s'appuie sur ce résultat conservé.

Automatiser les invariants

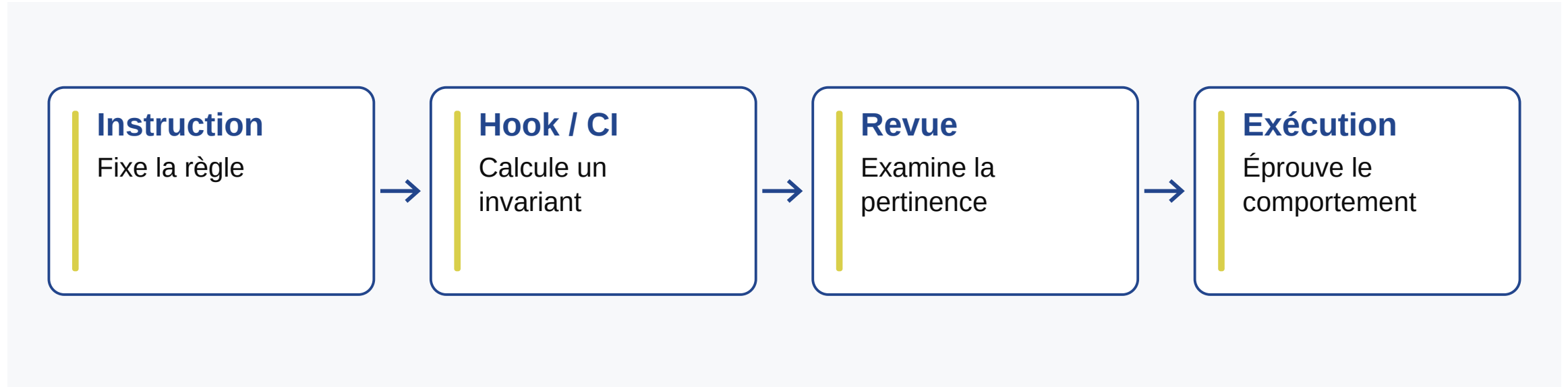
Un contrôle mécanique ne connaît pas votre intention.

- Identifier l'événement qui déclenche le contrôle.
- Lire les entrées réellement examinées.
- Distinguer forme, sens et comportement.



Distinguer contrôles mécaniques, CI, preuves et jugement.

Ce que chaque contrôle établit



Les entrées du contrôle déterminent la portée de son résultat.

Un exemple à examiner

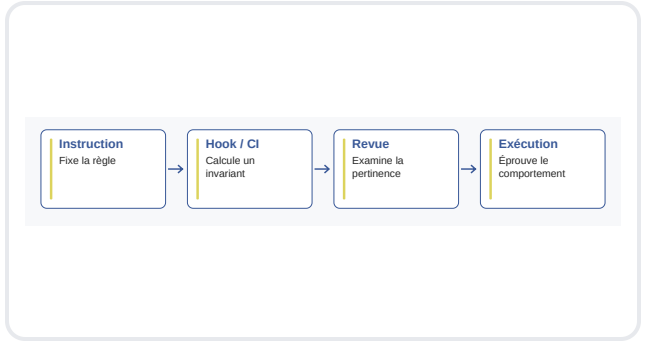
Marqueur ajouté après git add :
le fichier le contient, l'index non.
Hook rouge : examiner le blob indexé.
Ne pas désactiver le contrôle pour passer.



Prédire → observer → expliquer l'écart.

À vous de produire

- E25 · Cartographier les hooks et leurs limites.
- E26 · Analyser quatre incidents d'automatisation.
- Rendre événements, entrées, refus et paires de cas.

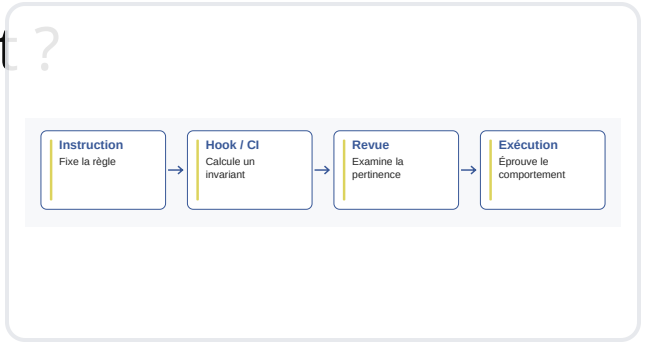


E25 + E26 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Une CI verte sur un index vide a-t-elle vérifié le changement ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

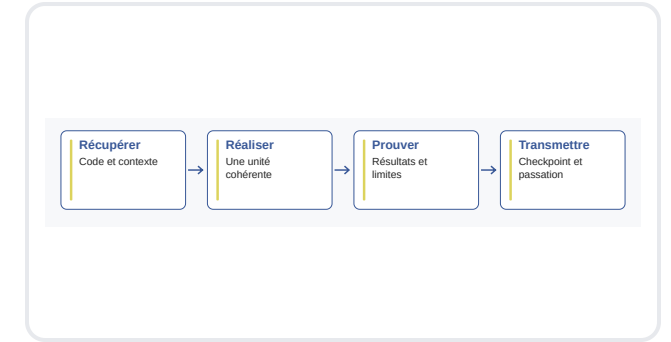


La reprise suivante s'appuie sur ce résultat conservé.

Passer au worker autonome

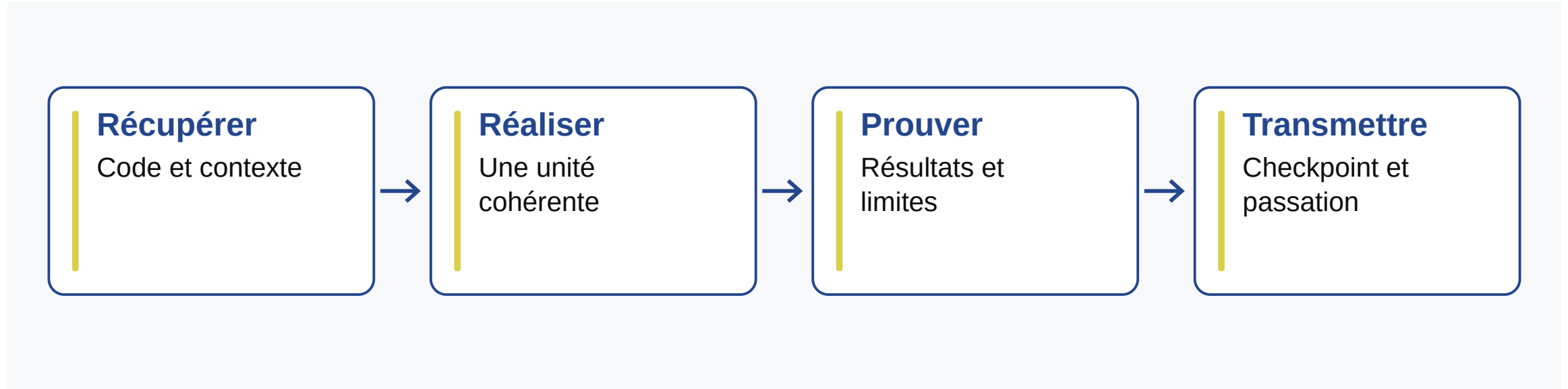
Une session autonome doit pouvoir être interrompue.

- Récupérer un état durable connu.
- Borner budget, pouvoirs et destination.
- Laisser une passation et une condition d'arrêt.



Construire une reprise après interruption avec limites et arrêt.

Une session qui laisse une reprise



Budget, autorité, destination et condition d'arrêt sont explicites.

Un exemple à examiner

Budget fictif : 30 minutes.

Code cohérent à la minute 16.

Interruption à 18 : preuves commencées.

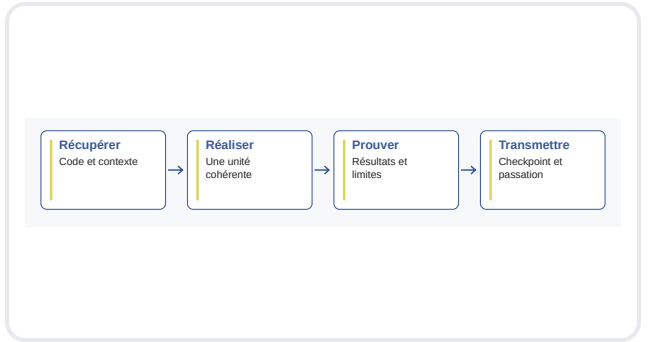
Checkpoint conservé ; unité encore en cours.



Prédire → observer → expliquer l'écart.

À vous de produire

- E27 · Spécifier le worker sur table.
- E28 · Interrompre, transmettre puis reprendre.
- Rendre budget, passation et arrêt sans faux effet externe.

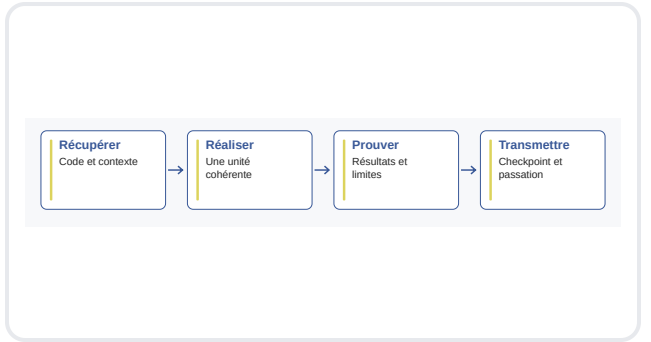


E27 + E28 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Le fichier docs/.routine installe-t-il une tâche planifiée ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.

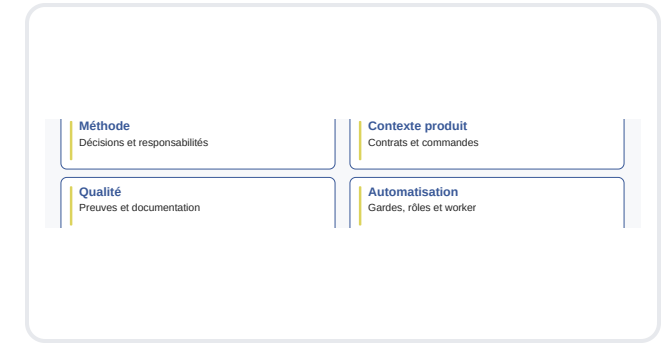


La reprise suivante s'appuie sur ce résultat conservé.

Assembler la software factory

Le socle final répond aux problèmes rencontrés avant lui.

- Assembler méthode, contexte, mémoire et preuves.
- Ajouter les contrôles utiles et les rôles bornés.
- Qualifier l'adoption sur une petite unité réelle.



Reconstruire la configuration finale et justifier chacune de ses pièces.

L'usine assemble les acquis

Méthode

Décisions et responsabilités

Contexte produit

Contrats et commandes

Qualité

Preuves et documentation

Automatisation

Gardes, rôles et worker

Le projet ajoute sa stack, ses données, ses accès et ses services réels.

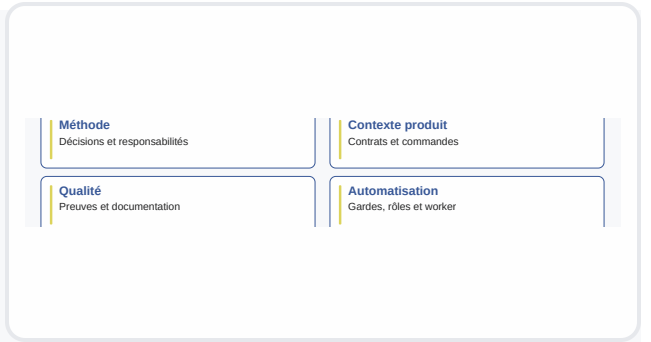
Un exemple à examiner

Besoin vécu → pièce justifiée

Reprise → documents et Git

Règle répétable → garde déterministe

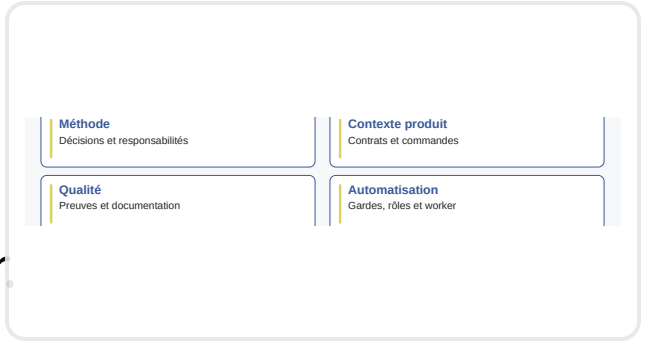
Automatisation → protocole + infrastructure



Prédire → observer → expliquer l'écart.

À vous de produire

- E29 · Reconstruire puis comparer l'assemblage.
- E30 · Préparer une adoption justifiée.
- Rendre une carte, des limites et une première unité à prouver.

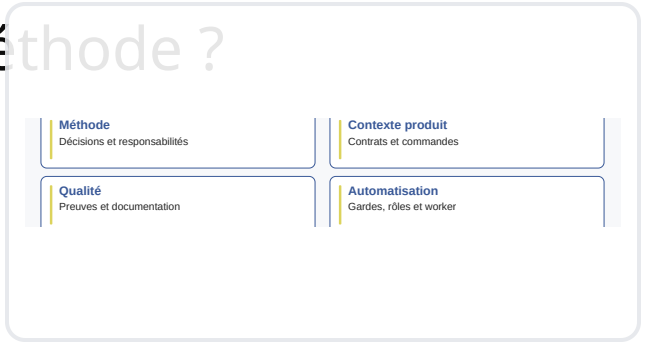


E29 + E30 · consignes complètes dans le cahier d'exercices

Vérifier votre compréhension

Quelle première preuve montre une adoption utile de la méthode ?

- Répondre avant d'ouvrir le corrigé.
- Expliquer un contre-exemple.
- Conserver le livrable et la prochaine action.



L'évaluation finale vérifie les gestes, pas la copie du socle.

Démontrer votre autonomie

DEM-03 : permettre au responsable de rouvrir une demande.

- Partir de la référence corrigée.
- Livrer contrat, code, preuves et passation.

270 minutes · individuel · critères connus avant de commencer

Prouver les succès et les refus

- Alice rouvre ; Bob et Eve sont refusés.
- Aucune mutation après refus ; répétition sans doublon.
- Parcours depuis l'accueil ; régressions rejouées.

Le corrigé de référence contient douze tests ; leur nombre n'est pas le barème.

Rendre un état défendable

- Cadrage 10 · contrat 15 · code 20.
- Preuves 30 · documentation 15 · revue 10.
- Seuil : 70/100, sans faute critique définie au barème.

Votre prochaine unité personnelle devient le premier transfert de la méthode.